

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO PHÁT TRIỂN ỨNG DỤNG IOT
BÀI THỰC HÀNH 1

Giảng viên : Nguyễn Quốc Uy

Sinh viên : Điền Ngọc Hải

MSV : B22DCPT071

Hà Nội – 2026

Contents

Phần I: Mô tả dự án.....	3
1. Giới thiệu.....	3
2. Mô tả tổng quát.....	3
3. Các yêu cầu chức năng hệ thống	3
4. Yêu cầu về giao diện.....	4
5. Luồng xử lý dữ liệu.....	4
Phần II: Thiết kế giao diện/ứng dụng	5
1. Dashboard.....	5
2. Sensor History	6
3. Device History	6
4. Profile.....	7
Phần III: Sơ đồ tuần tự	8
1. Dashboard - Đọc cảm biến - hiển thị data real time	8
2. Điều khiển thiết bị	9
3. Truy vấn Sensor History	11
4. Truy vấn Device History	13
Phần IV: Thiết kế cơ sở dữ liệu	15
1. Sơ đồ lớp	15
2. Sơ đồ cơ sở dữ liệu	16

Phần I: Mô tả dự án

1. Giới thiệu

Tài liệu này để xác định các yêu cầu và chức năng cho dự án hệ thống: “Giám sát môi trường (với 3 yếu tố: Nhiệt độ, độ ẩm và ánh sáng) và điều khiển thiết bị từ xa qua nền tảng Web”.

Phạm vi dự án: Hệ thống bao gồm thiết bị đầu cuối (ESP8266); hệ thống quản lý trung tâm (Back-end và Database); giao thức giao tiếp giữa thiết bị đầu cuối và hệ thống quản lý (MQTT và Mosquito broker); Giao diện người dùng (Front-end).

2. Mô tả tổng quát

Mô hình hoạt động: Hệ thống tuân thủ theo mô hình 4 lớp: Hardware (thiết bị) – MQTT – Backend – Frontend, đảm bảo cho khả năng điều khiển từ xa, cập nhập dữ liệu liên tục và khả năng mở rộng trong tương lai.

Ràng buộc về phần cứng:

- Vi điều khiển trung tâm: ESP8266
- Các cảm biến sử dụng: DHT11 (cho nhiệt độ và độ ẩm); LDR (cho ánh sáng)
- Thiết bị (giả lập): **Đèn chiếu sáng, Quạt và Điều hòa** (3 thiết bị kể trên sẽ được giả lập thay thế bằng 3 bóng LED với thứ tự tương ứng: LED 1, 2 và 3)

Ràng buộc về phần mềm:

- Giao thức truyền thông: MQTT (Mosquito broker) cho việc giao tiếp giữa thiết bị IoT và back-end và WebSocket cho việc update realtime data.
- Công nghệ sử dụng cho phát triển: ReactJS, NodeJS, Express và MySQL.

3. Các yêu cầu chức năng hệ thống

Giám sát thời gian thực:

- Yêu cầu: Hệ thống phải thu thập dữ liệu về nhiệt độ, độ ẩm, ánh sáng với chu kỳ 2 giây/lần.
- Hiện thị: Dữ liệu phải được cập nhập tức thời lên Dashboard dưới dạng số liệu và biểu đồ.

Điều khiển thiết bị: Chế độ thủ công (only): Cho phép người dùng bật tắt thiết bị từ giao diện Web.

Quản lý lịch sử thao tác thiết bị: Hệ thống phải lưu trữ toàn bộ giá trị cảm biến và lịch sử thay đổi trạng thái của thiết bị vào trong database. Cho phép người dùng xem, tìm kiếm và sắp xếp dữ liệu theo các tiêu chí như sau: loại cảm biến, giá trị, khoảng thời gian

4. Yêu cầu về giao diện

Giao diện người dùng: Phải bao gồm các trang sau: Dashboard, Sensor History, Device History, Profile. Trong đó:

- Dashboard: Coi như trang home của web, chứa tất cả các thông tin cơ bản liên quan. Ví dụ như tình trạng on/off các thiết bị, dữ liệu realtime thu được từ các sensor, graph về dữ liệu thu được qua thời gian của sensor.
- Sensor History: Trình bày tất cả các dữ liệu thu được từ sensor, được hiển thị ở dạng table (có xử lý phân trang), làm mới và cập nhập liên tục từ backend và database (theo chu kỳ 2 giây/lần).
- Device History: Trình bày tất cả dữ liệu về lịch sử thay đổi trạng thái của từng thiết bị trong hệ thống ở dạng table (và có xử lý phân trang).
- Profile: Giới hạn lại chỉ bao gồm thông tin người làm dự án (bản thân).

Giao diện truyền thông: Sử dụng giao thức MQTT để publish/subscribe dữ liệu giữa phần cứng và Backend qua 1 topic tự tạo.

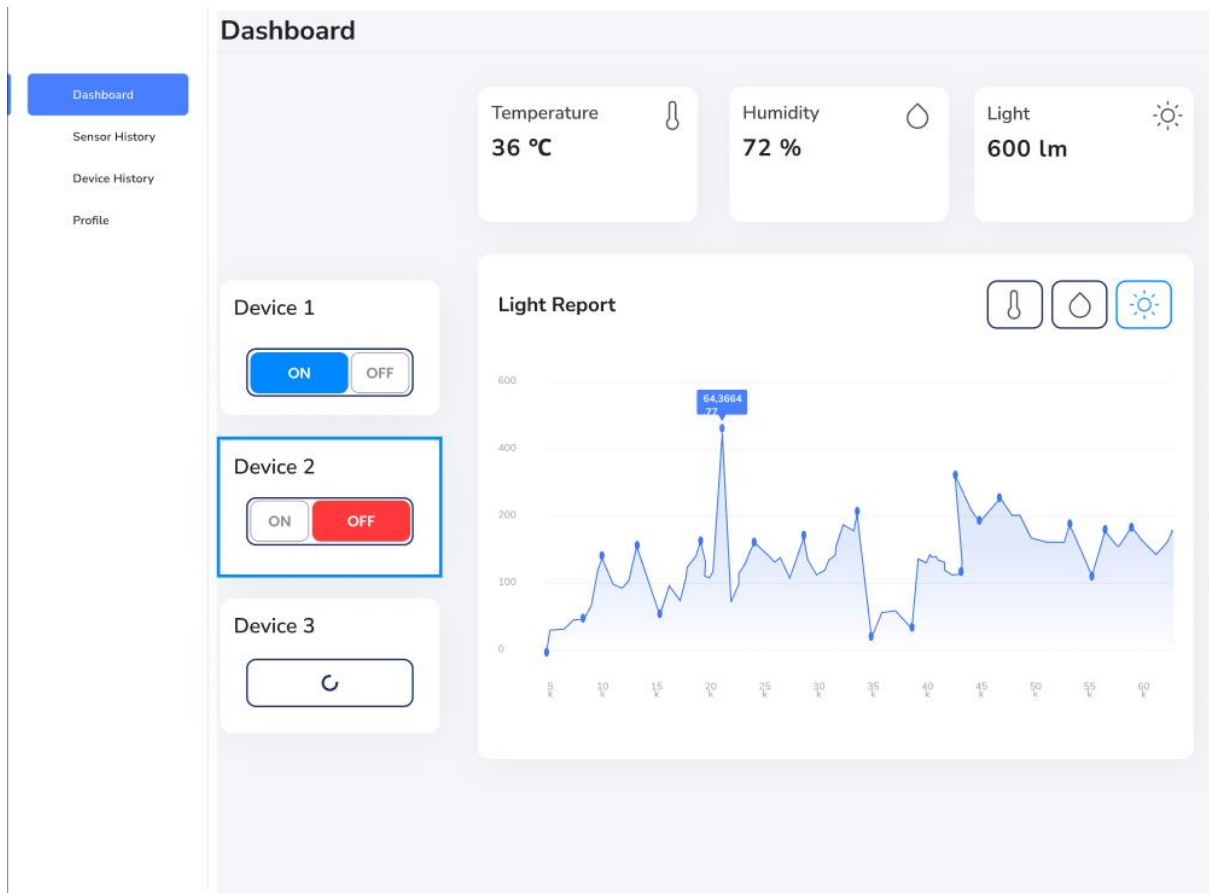
5. Luồng xử lý dữ liệu

Luồng tổng quan:

1. ESP8266 đọc dữ liệu từ cảm biến mỗi 2 giây.
2. Thiết bị được xử lý logic on/off tự động theo điều kiện môi trường (chế độ auto).
3. Dữ liệu cảm biến + Trạng thái thiết bị được publish lên MQTT Broker.
4. Backend subscribe vào topic qua MQTT, lấy dữ liệu và lưu dữ liệu vào trong database.
5. Backend đẩy dữ liệu realtime lên Frontend.
6. Dashboard cùng các trang liên quan cập nhập dữ liệu mới để hiển thị.

Phần II: Thiết kế giao diện/ứng dụng

1. Dashboard



2. Sensor History

Dashboard

Sensor History

Device History

Profile

Sensor History

Q Search

Date

Q

ID	Sensor	Value	Date
----	--------	-------	------

Rows per page: 1 6 of 10 < >

3. Device History

Dashboard

Sensor History

Device History

Profile

Device History

Q Search

Date

Q

ID	Device	Action	Status	Running	Date
----	--------	--------	--------	---------	------

Rows per page: 1 6 of 10 < >

4. Profile

Dashboard

Sensor History

Device History

Profile

Profile



Điền Ngọc Hải

📌 Công nghệ Đa phương tiện - Học viện Công nghệ Bưu chính Viễn thông

📅 26/09/2004

☎ 0337285594

✉ haidn2692004@gmail.com

📍 Hà Đông, Hà Nội

🗨️ haidayn_fhaidayyy

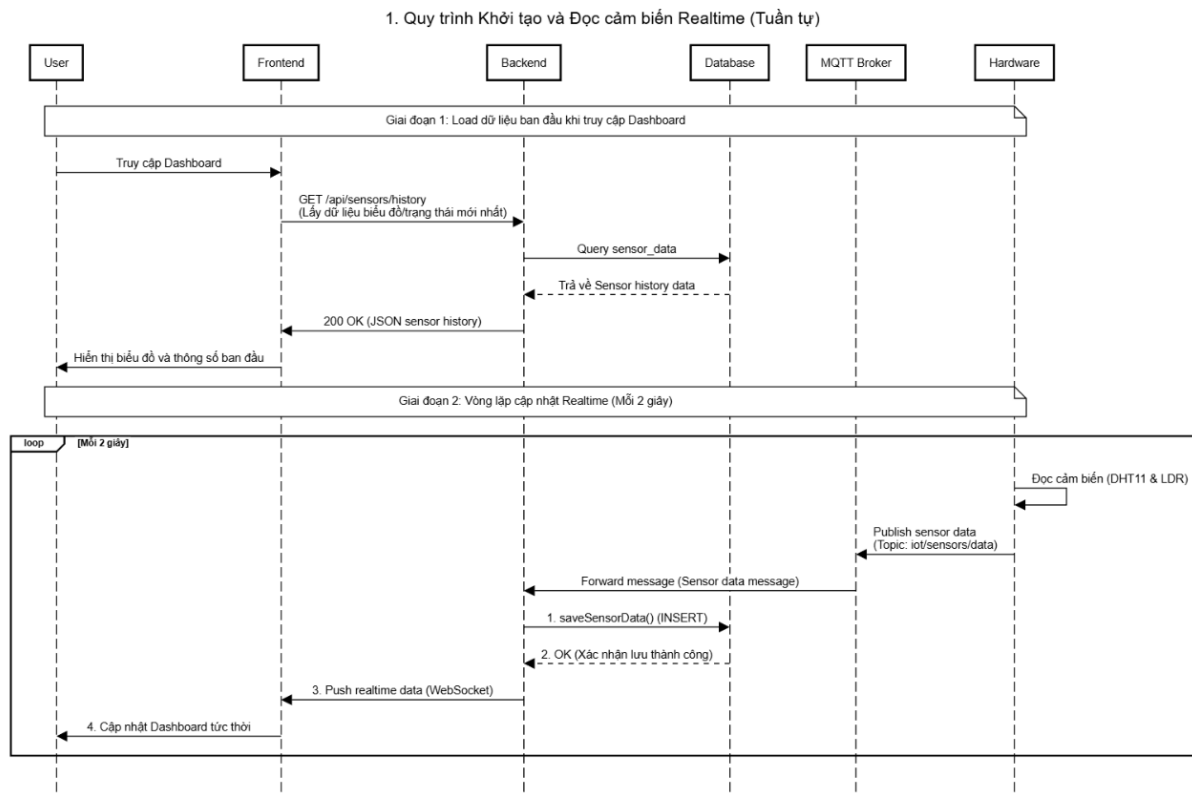
✉ haidn2692004@gmail.com

🌐 (B) Hải Điền | LinkedIn

🔗 https://www.figma.com/design/g803ILu8Euh...

Phần III: Sơ đồ tuần tự

1. Dashboard - Đọc cảm biến - hiển thị data real time



Luồng tổng quát:

[giai đoạn load data ban đầu]

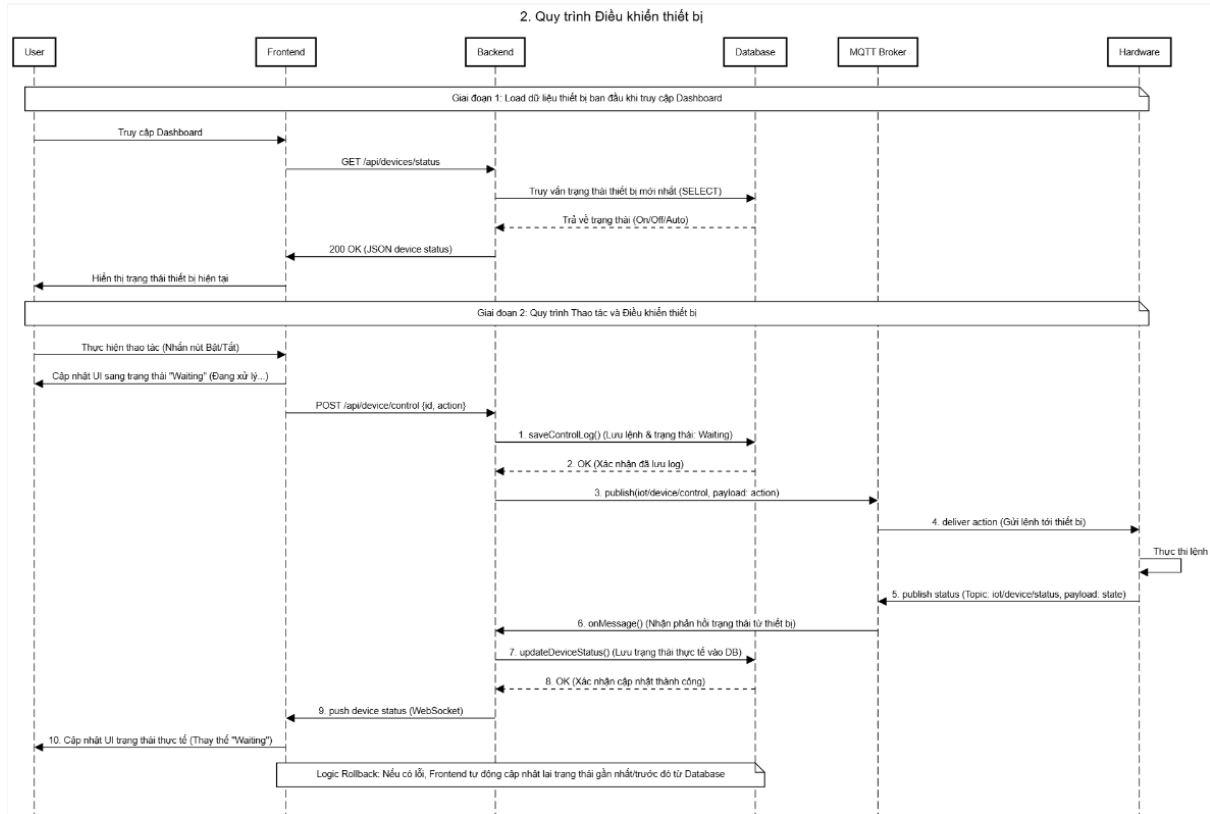
1. Ngay khi truy cập dashboard sẽ gửi yêu cầu GET tới backend API để lấy các giá trị bản ghi cũ.
2. Backend thực hiện truy vấn vào cơ sở dữ liệu MySQL để lấy danh sách lịch sử cảm biến và trạng thái thiết bị gần nhất.
3. Dữ liệu lịch sử sau đó được trả về cho Frontend dưới dạng JSON. Frontend sử dụng dữ liệu này để vẽ biểu đồ và hiển thị các con số trạng thái đầu tiên cho người dùng quan sát.

[giai đoạn loop realtime]

4. Định kỳ 2s/lần hardware sẽ đọc thông số từ cảm biến (DHT11, LDR) để gửi (publish) lên MQTT qua topic iot/sensors/data.
5. MQTT sẽ chuyển tiếp message này đến backend (backend đã subscribe vào topic trên: iot/sensors/data).
6. Backend thực hiện việc lưu data trên database.
7. Khi nhận được phản hồi lưu thành công, backend mới push data lên frontend qua WebSocket.

- Thực hiện việc cập nhật graph và info theo data nhận được từ backend trên frontend - dashboard.

2. Điều khiển thiết bị



Luồng tổng quát:

[giai đoạn load data ban đầu]

- Khi người dùng bắt đầu truy cập Dashboard, Frontend gửi yêu cầu GET tới Backend để lấy dữ liệu.
- Backend thực hiện lệnh SELECT để lấy trạng thái mới nhất của các thiết bị (Quạt, Điều hòa, Đèn) từ cơ sở dữ liệu.
- Dữ liệu được trả về Frontend dưới dạng JSON để giao diện hiển thị đúng trạng thái thực tế ngay từ đầu.

[quy trình thao tác và điều khiển thiết bị]

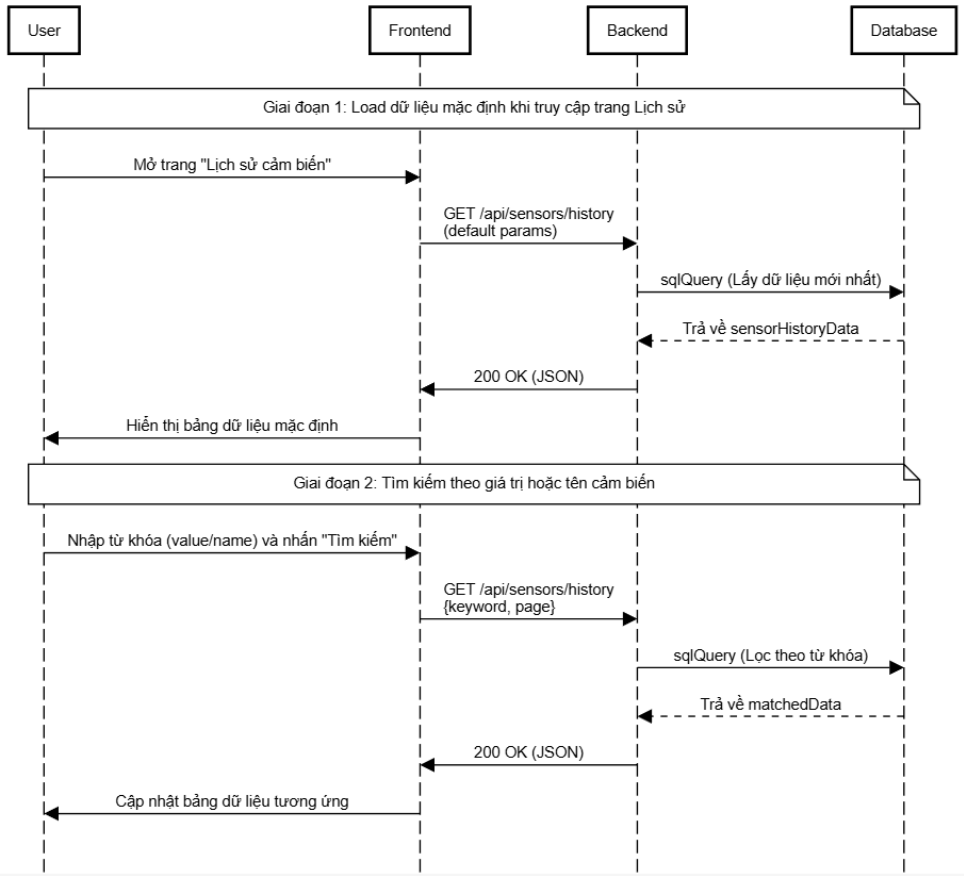
- Người dùng thực hiện nhấn nút On/Off hoặc thay đổi giá trị trên giao diện.
- Frontend lập tức chuyển đổi hiển thị của thiết bị đó sang trạng thái **"Waiting"** (Đang xử lý). Bước này giúp người dùng biết hệ thống đã ghi nhận lệnh và đang trong quá trình giao tiếp với phần cứng.

6. Frontend gửi yêu cầu POST /api/device/control kèm theo định danh thiết bị (deviceId) và hành động mong muốn (action) tới Backend.
7. Backend gọi hàm saveControlLog() để lưu thông tin lệnh điều khiển vào Database với trạng thái ban đầu là "Waiting".
8. Chỉ khi Database xác nhận đã lưu log thành công, Backend mới thực hiện publish() lệnh điều khiển lên MQTT Broker tại Topic iot/device/control.
9. MQTT Broker chuyển tiếp lệnh tới Hardware. Thiết bị thực hiện thao tác.
10. Sau khi thực hiện, Hardware gửi bản tin phản hồi trạng thái thực tế (deviceState) lên Topic iot/device/status thông qua MQTT Broker.
11. Backend nhận phản hồi từ Hardware, gọi hàm updateDeviceStatus() để ghi đè trạng thái thực tế vào Database.
12. Sau khi Database cập nhật thành công, Backend đẩy dữ liệu qua WebSocket tới Frontend.
13. Frontend nhận dữ liệu, thay thế trạng thái "Waiting" bằng trạng thái thực tế (ví dụ: chuyển từ "Waiting" sang "On").

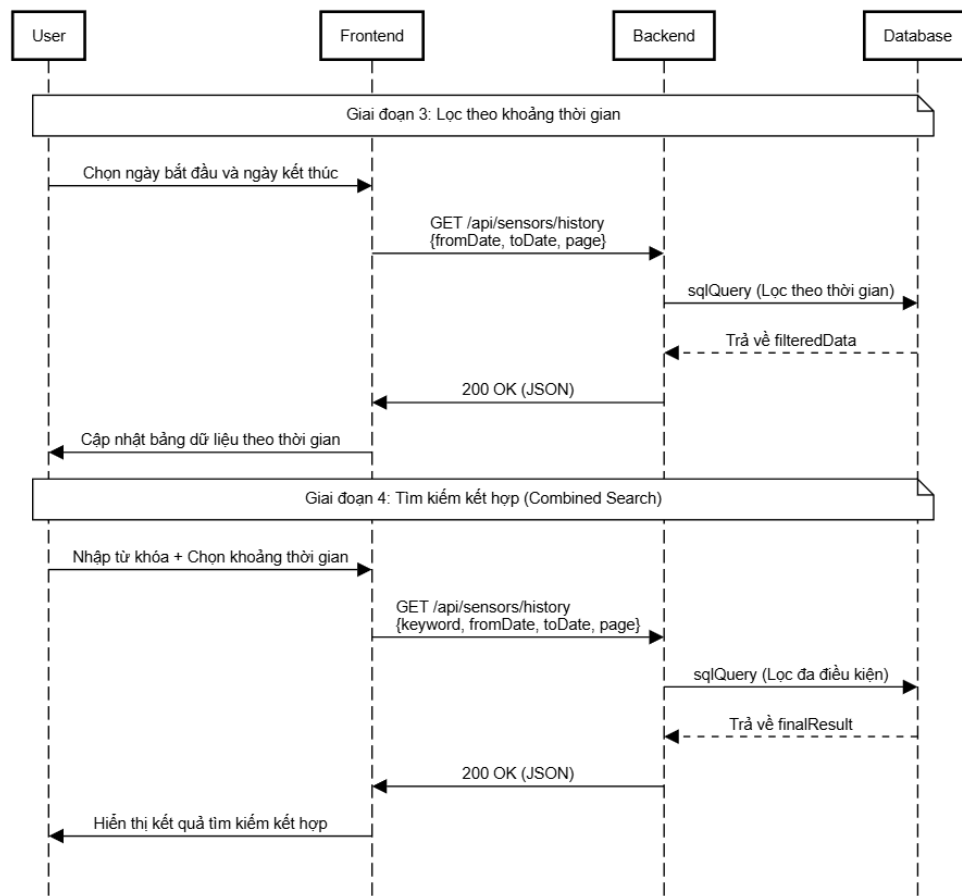
Cơ chế Rollback: Trong trường hợp xảy ra lỗi (mất kết nối, thiết bị không phản hồi), Frontend sẽ tự động truy vấn lại trạng thái hợp lệ gần nhất trong Database để cập nhật lại giao diện, tránh tình trạng hiển thị sai lệch so với thực tế.

3. Truy vấn Sensor History

3. Lịch sử cảm biến



3. Lịch sử cảm biến



Luồng tổng quát

[giai đoạn load data ban đầu]

1. Khi người dùng mở trang Sensor History - "Lịch sử cảm biến", **Frontend** gửi yêu cầu GET /api/sensors/history với các tham số mặc định (thường là trang 1 và không có bộ lọc).
2. **Backend** nhận yêu cầu và thực thi sqlQuery để lấy danh sách các bản ghi mới nhất từ Database.
3. Dữ liệu lịch sử được trả về và hiển thị ngay lập tức lên bảng giúp người dùng có thông tin tổng quan mà không cần thao tác thêm.

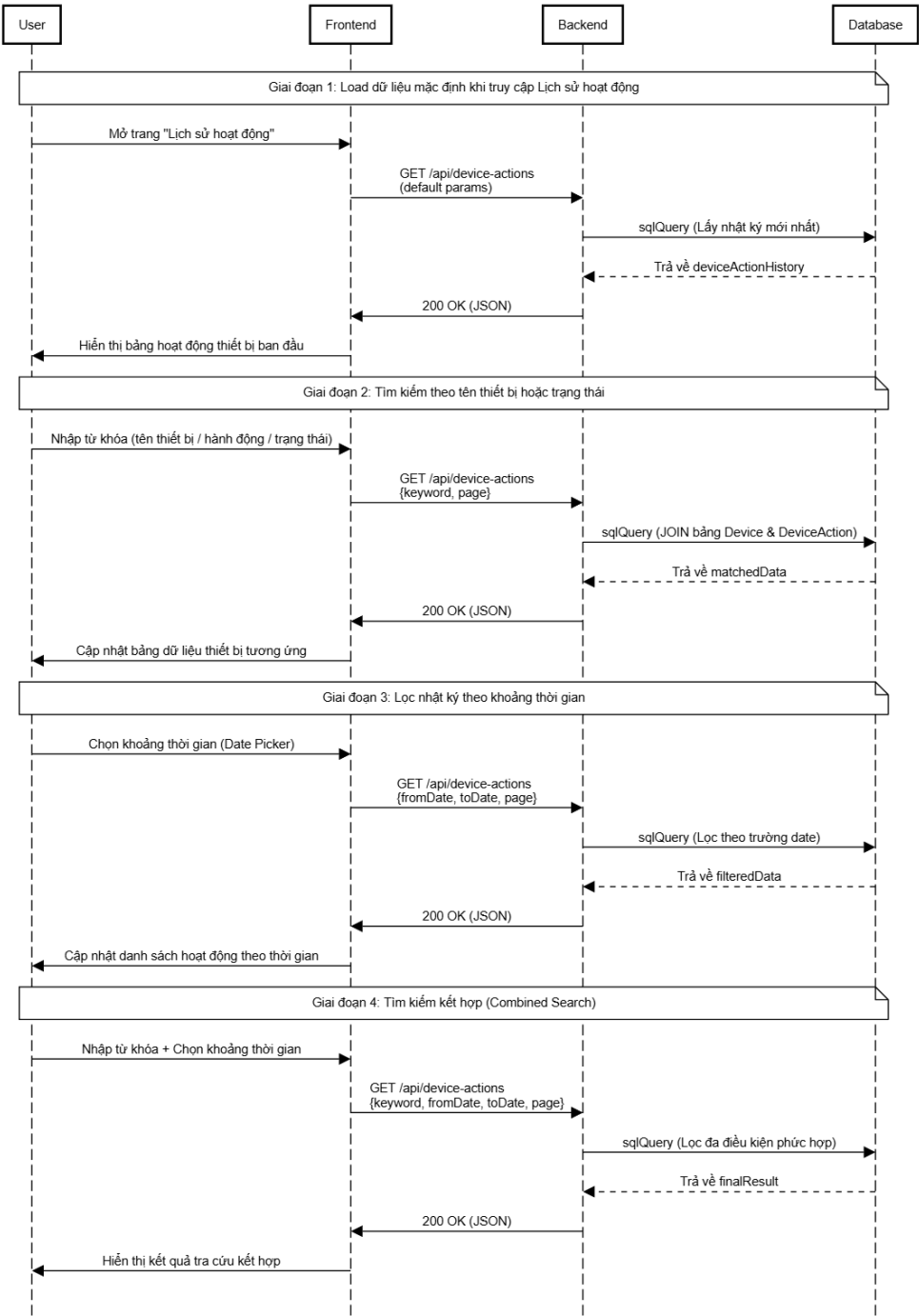
[thao tác lọc data riêng biệt]

4. User nhập từ khóa (ví dụ: "Nhiệt độ" hoặc giá trị "27") hoặc nhập ngày tháng vào ô tìm kiếm.
5. Hệ thống thực thi sqlQuery (tổng quan) với điều kiện lọc để trích xuất các dữ liệu được ghi nhận trong khoảng đó.
6. Kết quả trả về từ sqlQuery không chỉ bao gồm danh sách dữ liệu (data) mà còn có tổng số bản ghi (total) để phục vụ tính năng phân trang.

7. **Frontend** nhận phản hồi JSON, cập nhật lại trạng thái của bảng và render các dòng dữ liệu mới tương ứng với yêu cầu lọc của người dùng.

4. Truy vấn Device History

4. Quy trình Truy vấn Lịch sử thiết bị (Phân đoạn Filter)



Luồng tổng quát:

[giai đoạn load data ban đầu]

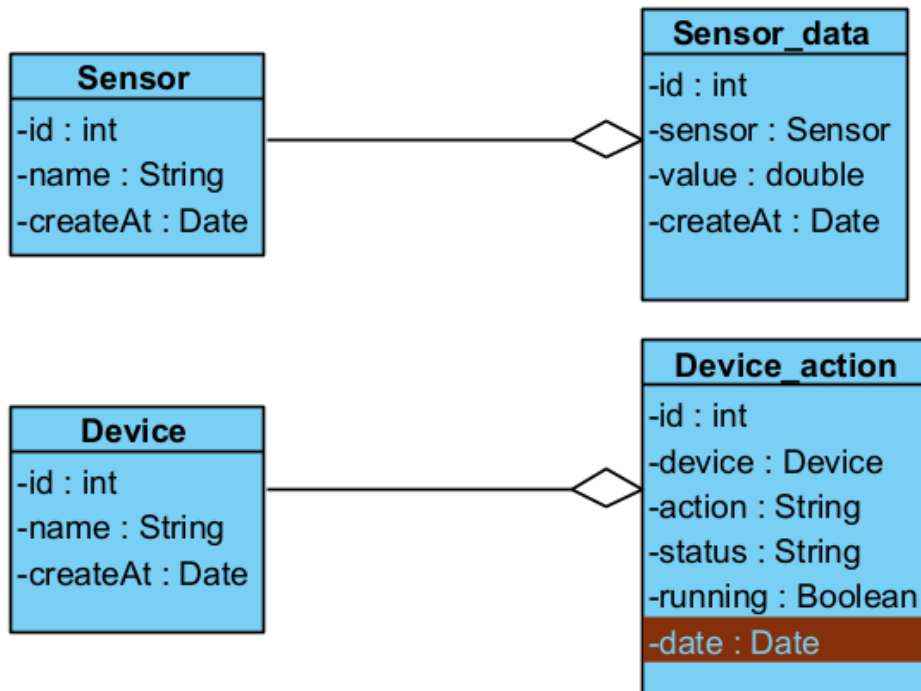
1. Khi người dùng mở trang Device History - "Lịch sử thiết bị", **Frontend** gửi yêu cầu GET /api/device-actions với các tham số mặc định (ví dụ: lấy 10 bản ghi mới nhất).
2. **Backend** nhận yêu cầu và thực thi sqlQuery để lấy danh sách các bản ghi mới nhất từ Database.
3. Dữ liệu được trả về và hiển thị lên bảng nhật ký, giúp người dùng thấy ngay các hoạt động gần đây của Quạt, Điều hòa hoặc Đèn.

[thao tác lọc data riêng biệt]

4. User nhập từ khóa (ví dụ: tên thiết bị như đèn, quạt,..)hoặc trạng thái hoạt động hoặc loại hành động (on/off) hoặc nhập ngày tháng vào ô tìm kiếm.
5. Hệ thống thực thi sqlQuery (tổng quan) với điều kiện lọc để trích xuất các dữ liệu được ghi nhận trong khoảng đó.
6. Kết quả trả về từ sqlQuery không chỉ bao gồm danh sách dữ liệu (data) mà còn có tổng số bản ghi (total) để phục vụ tính năng phân trang.
7. **Frontend** nhận phản hồi JSON, cập nhật lại trạng thái của bảng và render các dòng dữ liệu mới tương ứng với yêu cầu lọc của người dùng.

Phần IV: Thiết kế cơ sở dữ liệu

1. Sơ đồ lớp



2. Sơ đồ cơ sở dữ liệu

